CODE - A Pinguela 19-20

1.-Introducción

Git es un software de control de versiones diseñado por Linus Torvalds, pensando en la eficiencia y la confiabilidad del mantenimiento de versiones de aplicaciones cuando éstas tienen un gran número de archivos de código fuente. Su propósito es llevar registro de los cambios en archivos de ordenador y coordinar el trabajo que varias personas realizan sobre archivos compartidos.

JavaDoc en cambio es una utilidad de Oracle que nos permite la generación de documentación de un API a partir de anotaciones de nuestro código.

Ambas herramientas permiten la realización de grandes proyectos de desarrollo software.

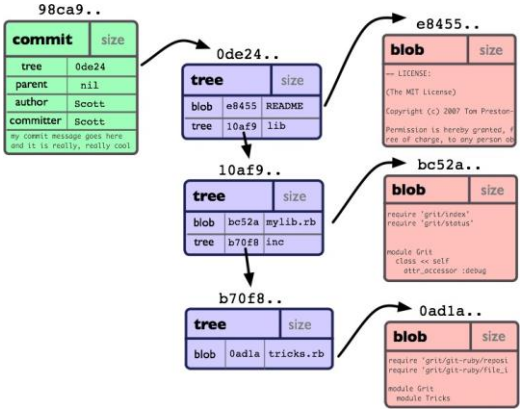
2.-Enunciado

Para realizar estos ejercicios es necesario que hagas uso de la cuenta de GitHub que has creado en clase. Por cada ejercicio es necesario que crees un repositorio en tu cuenta.

2.1 Ejercicio 1

Para entender cómo funciona GIT es necesario saber cómo lo hace por dentro. Para ello se necesita conocer estos términos: **SHA, blob, tree, commit, tag**.

Define estos términos conociendo la información que alberga cada uno y como se relacionan entre ellos para hacer funcionar un repositorio. Posteriormente explica lo que ve en esta imagen detallando información.



1. Clona estos repositorios en tu cuenta de Githb:

   https://github.com/jreycid/A-Pinguela-CODE---Tutorial-GIT.git

   https://github.com/jreycid/A-Pinguela-CODE---Ejemplo-completo-.gitignore.git

Responde a las siguientes preguntas:

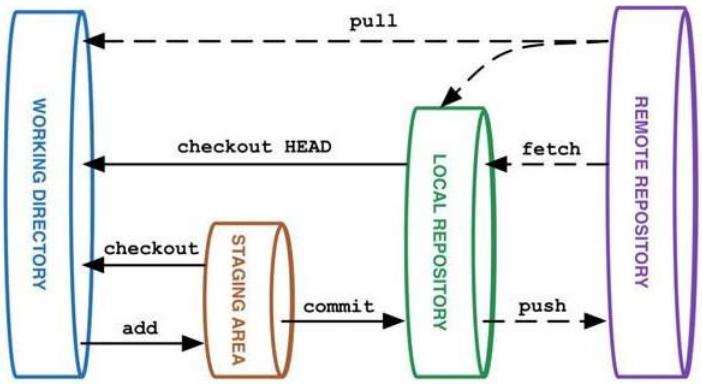
2. ¿Por qué agregar un archivo README.md a tu repositorio?
3. ¿Qué es Markdown?
4. Define los conceptos de “master”, “origin” y HEAD.
5. ¿Qué es y para que sirve un fork?
6. ¿Qué es y para que sirve el comando git rebase?.
7. ¿En que se diferencia de un git merge?
8. ¿Qué es y para que sirve un pull request?

Para facilitar la búsqueda de información se sugiere utilizar el siguiente enlace:
http://shafiul.github.io/gitbook/1_the_git_object_model.html

CODE - A Pinguela 19-20

2.1 Ejercicio 2

Utilizando el terminal de comandos con el programa GIT realiza el siguiente ejercicio. Escribe los comandos que has necesitado introducir en el terminal para llevara a cabo el ejercicio.



Busca las siguientes vistas. Realiza una captura de pantalla de cada una y sitúala en los recuadros.

1. Crea 3 archivos con cualquier contenido: archivo1.txt, archivo2.txt y archivo3.txt. Estos archivos están contenidos en sus respectivos directorios directorio1, directorio2 y directorio3.
2. Inicializa un repositorio local
3. Añade al stage los archivos de los directorios 1 y 2.
4. Comprueba con git status que el stage tiene lo que quieres.
5. Crea un commit con esos dos archivos.
6. Realiza una modificación en el texto del archivo 1/1.txt y otra en el texto del archivo 2/2.txt.
7. Añade al stage únicamente las modificaciones del archivo 2/2.txt.
8. Haz git status e interpreta la información que te da.
9. Crea un commit con las modificaciones que has hecho en el archivo 2/2.txt y con el archivo 3/3.txt.
10. Haz git status y comprueba que el stage está vacío y que en el working directory está el cambio del archivo 1/1.txt.
11. Desversiona el archivo 1/1.txt (es decir, dile a Git que no lo versione), pero sin eliminarlo.
12. Elimina el archivo 2/2.txt.
13. Haz git status y comprueba que la eliminación de los dos archivos está en el stage.
14. Crea un commit que almacene la eliminación de esos dos últimos archivos.
15. Haz git status y comprueba que todo está como esperas (el archivo 1/1.txt está sin versionar y el stage está sin cambios).
16. Configura el repositorio para que ignore la carpeta 1.
17. Haz git status y comprueba la diferencia.

CODE - A Pinguela 19-20

2.1 Ejercicio 3

Para este ejercicio es necesario montar un repositorio conjunto. Es necesario hacerlo en grupos de 3. Lo primero es por ello en GitHub montar un equipo de trabajo de 3 personas y crear un repositorio. Todas ellas tienes que tener todos los poderes sobre el repositorio.

Cada integrante deberá seguir los pasos de los ficheros que se dejan a continuación. Finalmente detalla todos los pasos realizados por cada miembro.

A la hora de escribir el código emplea las anotaciones de JavaDoc. Describe la función de la clase o del interfaz además de las funciones y sus diversos parámetros. Finalmente genera la documentación en formato HTML del todo el proyecto empleando IntelliJ. Usa el menú Tools la opción Generar JavaDocs.

Alumno A:

1. Crea un proyecto Java con nombre facturas y este súbelo al repositorio remoto a la rama Master.
2. Crea una rama "alumno A" para trabajar tu.
3. Se te ha encargado crear una interfaz que será implementada por todas las facturas.
4. Crea un interfaz llamado IFactura.java, con tres métodos:

```
void cambiarEstado(Estados estado);
float getTotal();
float getCantidadIva();
```

5. Crea una clase enumerado llamado Estados.java, que se llame Estados y que tenga como valores Aprobada, Pendiente y Cobrada.
6. Crea un commit para añadir el enum.
7. Crea un commit para añadir la interfaz.
8. Vuelca el trabajo a la rama Master y borra la rama que has usado.
9. Cuando tengas en tu repositorio la clase Factura que ha implementado tu compañero, crea una rama para corregir un bug referente a la propiedad estado de la clase Factura.
10. Corrige el bug: cambia la visibilidad de public a private. Haz un commit con esta corrección.
11. Vuelca la modificación a la rama Master y borra la rama que has usado.
12. Cuando esté lista la primera puesta en producción mediante la rama master, bájatela a tu repositorio local.

CODE - A Pinguela 19-20

Alumno B

1. Se te ha encargado empezar el esqueleto de la clase Factura. Crea una rama para ello llamada “Alumno B” a partir de la versión actualizada de Master.
2. Crea un archivo llamado Factura.java para la clase Factura, con los siguientes miembros además del método main():

```
public int num;
public float base;
public float tipoIva;
public Estados estado;
```

3. Comenta la línea de la propiedad "estado", porque aún no está implementado el enum Estados.
4. Crea un commit con la clase.
5. Vuelca el trabajo a la rama Master y borra la rama que has usado.
6. Cuando tengas la interfaz y el enum (desarrollada por otro compañero) en tu repositorio local, crea una rama para aplicar los siguientes cambios.
7. Descomenta la línea que comentaste en el punto 5 y haz un commit con esta modificación.
8. Implementa la interfaz en tu clase Factura (no hace falta que implementes los métodos, déjalos en blanco).
9. Crea un commit con este último cambio.
10. Corrige un bug: cambia la visibilidad de la propiedad estado a protected. Haz un commit con esta corrección. Si más adelante hay un conflicto con algún compañero del equipo, ésta es la visibilidad que tiene que quedarse definitivamente.
11. Vuelca el trabajo a la rama Master y borra la rama que has usado.
12. Eres el encargado de preparar la primera versión que irá a producción:
13. Añade un tag al último commit de master con la versión 0.1.0.

Alumno C

1. Se te ha encargado implementar algunos métodos de la clase Factura y un enum de estados. Empieza creando una rama llamada “Alumno C” para el enum a partir de la versión actualizada de Master.
2. Crea un archivo llamado Estados.java para un enum, que se llame Estados y que tenga como valores Aprobada, Pendiente, Cobrada y Anulada.
3. Si más adelante aparece este mismo enum en la rama de algún compañero del equipo y hay conflictos, éstos son los cuatro valores definitivos que tienen que quedar.
4. Crea un commit para añadir el enum.
5. Vuelca el trabajo a la rama Master y borra la rama que has usado.
6. Cuando tus compañeros hayan añadido estos dos métodos a la clase Factura, impleméntalos:

```
getTotal();
getCantidadIva();
```

7. Crea un commit con los cambios.
8. Vuelca el trabajo a la rama Master y borra la rama que has usado.
9. Sube los cambios de develop al repositorio remoto.
10. Cuando esté lista la primera puesta en producción mediante la rama master, bájatela a tu repositorio local.